

# BỘ CÂU HỎI ÔN TẬP TOÀN DIỆN

## Vị trí: DATA ENGINEER

DAC Data Technology Vietnam – Hakuhodo DY One

Da Nang Office | Full-time | Tháng 9 / 2026

|                 |                      |
|-----------------|----------------------|
| Tổng số câu hỏi | 537                  |
| Số chủ đề       | 23                   |
| Cấp độ          | Junior Senior        |
| Ngôn ngữ        | Tiếng Việt + English |

Tài liệu được tổng hợp từ JD chính thức của DAC Data Technology Vietnam. Câu hỏi bao phủ toàn bộ kỹ năng: Python, Java, SQL, GCP, ETL, CI/CD, Looker, AWS, Big Data, System Design, Security, MLOps, Domain Knowledge và Soft Skills.

# MỤC LỤC

---

- I. Python cho Data Engineering (40 câu)
- II. Java cho Data Engineering (15 câu)
- III. SQL & Database Systems (40 câu)
- IV. Data Modeling & Architecture (30 câu)
- V. ETL / ELT & Data Pipeline (40 câu)
- VI. Google Cloud Platform (GCP) (40 câu)
- VII. Git, GitFlow & CI/CD (25 câu)
- VIII. Looker & Data Visualization (20 câu)
- IX. AWS, Redshift & Treasure Data (20 câu)
- X. Big Data & Distributed Systems (30 câu)
- XI. System Design cho Data Engineer (25 câu)
- XII. API Integration & Web Services (20 câu)
- XIII. Monitoring, Observability & Performance (20 câu)
- XIV. Security & Compliance trong Data Engineering (20 câu)
- XV. Containers & Infrastructure (15 câu)
- XVI. Data Quality & Testing (15 câu)
- XVII. Machine Learning Engineering cho Data Engineer (15 câu)
- XVIII. Networking & Distributed Systems Fundamentals (10 câu)
- XIX. Quảng cáo & Marketing Technology (Domain Knowledge) (20 câu)
- XX. Soft Skills & Behavioral Questions (25 câu)
- XXI. Japanese & English Communication (12 câu)
- XXII. Câu hỏi tình huống & Problem Solving (20 câu)
- XXIII. Coding Challenges & Algorithms (20 câu)

Tổng cộng: 537 câu hỏi / 23 chủ đề

## I. Python cho Data Engineering

- Câu 1.** Giải thích sự khác biệt giữa list, tuple, set và dict. Khi nào nên dùng từng loại trong data pipeline?
- Câu 2.** Generator expression vs list comprehension: khác nhau về bộ nhớ như thế nào? Cho ví dụ xử lý file 10 GB.
- Câu 3.** Viết hàm Python đọc file CSV lớn (> 10 GB) theo từng chunk mà không load toàn bộ vào RAM.
- Câu 4.** Context manager (`with` statement): Viết context manager tùy chỉnh bằng class và bằng `@contextmanager`.
- Câu 5.** Phân biệt `multiprocessing` và `multithreading`. Khi nào dùng cái nào cho tác vụ data engineering?
- Câu 6.** GIL (Global Interpreter Lock) là gì? Ảnh hưởng đến xử lý song song như thế nào? Cách vượt qua GIL?
- Câu 7.** Giải thích `asyncio` và `async/await`. Cho ví dụ gọi 50 API endpoint đồng thời dùng `asyncio.gather`.
- Câu 8.** Viết decorator `@retry(max\_attempts, delay)` tự động thử lại hàm khi gặp exception.
- Câu 9.** Viết decorator `@timer` đo thời gian thực thi của bất kỳ hàm nào.
- Câu 10.** Pandas vs Polars vs Dask: So sánh hiệu năng, memory usage và use-case phù hợp cho từng thư viện.
- Câu 11.** Viết code Python dùng Pandas để: đọc nhiều CSV, nối lại, xử lý missing values, ghi ra Parquet.
- Câu 12.** Giải thích `\*args` và `\*\*kwargs`. Viết hàm tổng quát nhận cấu hình kết nối database qua `\*\*kwargs`.
- Câu 13.** Type hints trong Python 3: Lợi ích khi dùng trong data pipeline. Cho ví dụ với `TypedDict`, `Optional`, `Union`, `Literal`.
- Câu 14.** Viết script kết nối GCS dùng `google-cloud-storage`, đọc và ghi file Parquet.
- Câu 15.** Dùng `pydantic` để validate schema một record từ API bên ngoài. Xử lý validation error như thế nào?
- Câu 16.** Giải thích Python memory management: reference counting, garbage collection, `\_\_del\_\_`. Cách profile memory dùng `memory\_profiler`.
- Câu 17.** Phân biệt `deepcopy` và `shallowcopy`. Khi nào cần deepcopy trong data processing?
- Câu 18.** Giải thích `\_\_slots\_\_` trong Python class. Lợi ích về memory khi dùng trong data models?
- Câu 19.** Viết Python script đọc dữ liệu từ PostgreSQL dùng `psycopg2` hoặc `sqlalchemy`, transform và ghi vào BigQuery.
- Câu 20.** Giải thích `functools.lru\_cache` và `functools.cache`. Khi nào dùng cache trong data pipeline?
- Câu 21.** Viết một class `DataPipeline` với phương thức `extract()`, `transform()`, `load()` theo pattern Template Method.
- Câu 22.** Giải thích Pythonic code: list comprehension, ternary operator, unpacking, walrus operator (`:=`). Cho ví dụ từng cái.
- Câu 23.** Dùng `concurrent.futures.ThreadPoolExecutor` và `ProcessPoolExecutor` để song song hóa I/O-bound và CPU-bound tasks.
- Câu 24.** Viết pipeline đơn giản dùng Python `queue.Queue` để producer-consumer pattern xử lý records.
- Câu 25.** Giải thích `dataclasses` và so sánh với `namedtuple`, `TypedDict`, và `pydantic BaseModel`. Khi nào dùng cái nào?

- Câu 26.** Python logging best practices trong data pipeline: structured logging, log levels, log rotation, tích hợp với Cloud Logging.
- Câu 27.** Viết unit test cho hàm transform dữ liệu dùng `pytest` và `pytest-mock`. Cách mock external dependencies.
- Câu 28.** Giải thích `pathlib.Path` so với `os.path`. Tại sao `pathlib` được ưa chuộng hơn trong Python hiện đại?
- Câu 29.** Dùng `argparse` hoặc `click` để tạo CLI tool cho ETL script, nhận các tham số như `--date`, `--env`, `--dry-run`.
- Câu 30.** Giải thích `\_\_init\_\_.py`, `\_\_main\_\_.py`, và cấu trúc thư mục Python package chuẩn cho data engineering project.
- Câu 31.** Viết hàm tính checksum (MD5/SHA256) của file để verify data integrity sau khi transfer.
- Câu 32.** Dùng `boto3` (AWS SDK) và `google-cloud-bigquery` trong cùng một script để migrate data từ S3 sang BigQuery.
- Câu 33.** Giải thích Python `typing.Protocol` và duck typing. Cách dùng Protocol để định nghĩa interface cho connector classes.
- Câu 34.** Viết code dùng `aiohttp` để gọi REST API async, xử lý rate limiting (429), và save response vào file JSONL.
- Câu 35.** Explain the difference between `is` and `==` in Python. When can this cause bugs in data processing?
- Câu 36.** Viết hàm `flatten\_dict` chuyển nested dict `{ 'a': { 'b': { 'c': 1 } } }` thành `{ 'a.b.c': 1 }`. Dùng để xử lý JSON từ API.
- Câu 37.** Giải thích Python `abc` module (Abstract Base Classes). Thiết kế abstract class cho các loại data source khác nhau.
- Câu 38.** Viết script Python tự động detect encoding của file text (UTF-8, UTF-16, Latin-1) trước khi đọc.
- Câu 39.** Giải thích `contextlib.suppress`, `contextlib.redirect\_stdout`. Ứng dụng trong testing và logging?
- Câu 40.** Dùng `pandas.DataFrame.pipe()` để xây dựng transformation pipeline dạng function chaining. Ưu điểm gì?

## II. Java cho Data Engineering

- Câu 41.** Giải thích sự khác biệt giữa `HashMap`, `LinkedHashMap`, `TreeMap`, `ConcurrentHashMap`. Khi nào dùng cái nào?
- Câu 42.** Java Streams API: Viết stream pipeline filter record log, extract field, group by, count dùng `Collectors.groupingBy`.
- Câu 43.** Giải thích Java memory model: heap, stack, metaspace. Cách tune JVM (`-Xms`, `-Xmx`, `-XX:+UseG1GC`) cho ứng dụng Big Data?
- Câu 44.** Giải thích Java garbage collectors: G1GC, ZGC, Shenandoah. Khi nào chọn ZGC?
- Câu 45.** Checked vs unchecked exceptions trong Java. Best practice xử lý exception trong data pipeline?
- Câu 46.** Giải thích `Optional` trong Java 8. Cách dùng để tránh NullPointerException trong data processing?
- Câu 47.** Java `CompletableFuture`: Viết ví dụ gọi nhiều API đồng thời và combine kết quả. So sánh với Python asyncio.

- Câu 48.** Giải thích `Comparable` và `Comparator` interface. Cách sort custom objects trong stream?
- Câu 49.** Generics trong Java: bounded wildcards (`? extends T`, `? super T`). Tại sao cần trong collection processing?
- Câu 50.** Apache Kafka Java client: Producer và Consumer API. Cách implement consumer group với auto offset commit?
- Câu 51.** Viết Flink job đơn giản đọc từ Kafka topic, aggregate theo window 5 phút, ghi kết quả ra BigQuery.
- Câu 52.** Giải thích Java `ExecutorService`, `ScheduledExecutorService`. Cách tạo thread pool cho parallel ETL?
- Câu 53.** Lombok: Giải thích các annotation thường dùng (`@Data`, `@Builder`, `@Slf4j`). Ưu/nhược điểm?
- Câu 54.** Giải thích design pattern Builder trong Java. Áp dụng khi xây dựng complex query/config object?
- Câu 55.** Maven vs Gradle: Phân biệt và khi nào chọn Gradle cho data engineering project?

### III. SQL & Database Systems

- Câu 56.** Viết SQL tính rolling 7-day average của doanh thu theo ngày từ bảng `orders(order_date, revenue)`.
- Câu 57.** Giải thích Window Functions: `ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`, `LAG()`, `LEAD()`, `NTILE()`. Cho ví dụ từng hàm.
- Câu 58.** CTE vs Subquery vs Temporary Table: So sánh hiệu năng và readability. Khi nào dùng recursive CTE?
- Câu 59.** Viết SQL tìm session từ event logs: session là chuỗi sự kiện cùng user, không có khoảng trống > 30 phút.
- Câu 60.** Giải thích EXPLAIN ANALYZE trong PostgreSQL. Đọc query plan: Seq Scan, Index Scan, Hash Join, Nested Loop?
- Câu 61.** Index B-tree, Hash, GIN, GiST, Partial, Covering Index. Khi nào tạo composite index? Tại sao index làm WRITE chậm hơn?
- Câu 62.** Partitioning trong BigQuery: ingestion time, column partition, clustering. Ảnh hưởng đến chi phí query?
- Câu 63.** Viết BigQuery SQL dùng `UNNEST` xử lý ARRAY và STRUCT. Ví dụ với bảng dữ liệu quảng cáo.
- Câu 64.** Normalization 1NF, 2NF, 3NF, BCNF. Tại sao data warehouse thường denormalize?
- Câu 65.** ACID properties: Atomicity, Consistency, Isolation, Durability. Phân biệt với BASE trong NoSQL.
- Câu 66.** Transaction isolation levels: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE và các anomaly.
- Câu 67.** Slowly Changing Dimension (SCD) Type 1, 2, 3, 4, 6. Implement SCD Type 2 trong BigQuery.
- Câu 68.** Viết BigQuery SQL deduplicate records, giữ bản ghi mới nhất theo `updated_at` với mỗi `id`.
- Câu 69.** BigQuery cost optimization: slot reservation, BI Engine, materialized views, partitioning best practices.
- Câu 70.** INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF JOIN. Khi nào CROSS JOIN gây vấn đề hiệu năng nghiêm trọng?
- Câu 71.** Viết SQL tính cohort retention: tỷ lệ user quay lại theo tháng kể từ tháng đầu tiên dùng sản phẩm.

- Câu 72.** MERGE (UPSERT) vs INSERT OVERWRITE. Dùng MERGE trong BigQuery như thế nào? Khi nào prefer INSERT OVERWRITE?
- Câu 73.** Database Sharding và Replication: horizontal sharding, read replicas, leader-follower, multi-master. Trade-offs?
- Câu 74.** Temporal tables / bi-temporal data: lưu lịch sử thay đổi dữ liệu. Áp dụng trong BigQuery?
- Câu 75.** Viết SQL phát hiện anomaly: giá trị outlier dùng IQR hoặc Z-score ngay trong BigQuery SQL.
- Câu 76.** Viết SQL tính \*percentile\* (p50, p90, p99) của latency từ bảng request logs dùng `APPROX\_QUANTILES` trong BigQuery.
- Câu 77.** Giải thích PIVOT và UNPIVOT trong SQL. Viết query chuyển rows thành columns cho báo cáo.
- Câu 78.** Viết SQL tìm chuỗi liên tiếp (consecutive days) user đăng nhập dùng window functions.
- Câu 79.** Giải thích Materialized View: khác thế nào với regular view? Khi nào dùng trong BigQuery?
- Câu 80.** Viết BigQuery SQL tính \*market basket analysis\*: những sản phẩm nào thường được mua cùng nhau?
- Câu 81.** Giải thích GROUPING SETS, ROLLUP, CUBE. Cho ví dụ tính tổng doanh thu theo nhiều chiều.
- Câu 82.** Viết SQL detect circular reference trong bảng có quan hệ parent-child (ví dụ: org chart).
- Câu 83.** Giải thích \*query optimizer\* trong BigQuery. Tại sao đôi khi viết query 'ngây thơ' lại nhanh hơn query 'tối ưu tay'?
- Câu 84.** Viết SQL tính \*funnel conversion rate\*: từ impression click add\_to\_cart purchase.
- Câu 85.** Giải thích `QUALIFY` clause trong BigQuery / Snowflake. Thay thế cho subquery với window function như thế nào?
- Câu 86.** Viết SQL tạo calendar dimension table với tất cả ngày trong năm, flag weekend/holiday, quarter, week number.
- Câu 87.** Giải thích \*approximate aggregations\* trong BigQuery: `APPROX\_COUNT\_DISTINCT`, `APPROX\_TOP\_COUNT`. Khi nào dùng?
- Câu 88.** Viết stored procedure / scripting trong BigQuery để tự động partition table theo tháng.
- Câu 89.** Giải thích column encryption trong BigQuery: AEAD functions. Mã hóa dữ liệu nhạy cảm ngay trong SQL.
- Câu 90.** Viết SQL tính \*time-to-first-purchase\* cho mỗi user: khoảng cách từ lần đầu tiên thấy quảng cáo đến lần mua đầu tiên.
- Câu 91.** Giải thích difference giữa `COUNT(1)`, `COUNT(\*)`, `COUNT(col)`. Cái nào nhanh hơn trong BigQuery?
- Câu 92.** Viết query dùng `WITH RECURSIVE` để traverse tree structure (ví dụ: comment thread, org chart).
- Câu 93.** Giải thích \*query cache\* trong BigQuery. Khi nào cache được dùng và khi nào không?
- Câu 94.** Viết SQL tính \*longest streak\*: chuỗi ngày dài nhất mà user liên tục hoạt động.
- Câu 95.** Giải thích `SAFE\_CAST`, `SAFE\_DIVIDE`, `COALESCE`, `NULLIF`, `IF`, `IFF` trong BigQuery. Tại sao quan trọng trong production?

## IV. Data Modeling & Architecture

- Câu 96.** Star Schema vs Snowflake Schema vs Data Vault: So sánh ưu/nhược điểm. Khi nào chọn từng loại?

- Câu 97.** Fact Table và Dimension Table: Thiết kế star schema cho hệ thống quảng cáo online.
- Câu 98.** Data Vault 2.0: Hubs, Links, Satellites. Ưu điểm trong môi trường agile và schema evolution?
- Câu 99.** Medallion Architecture (Bronze/Silver/Gold): mô tả data flow và transformations qua từng tầng.
- Câu 100.** Lambda Architecture vs Kappa Architecture: Phân biệt, ưu/nhược, khi nào chọn cái nào?
- 
- Câu 101.** Data Mesh: 4 nguyên tắc. Áp dụng ở công ty vừa và nhỏ có thực tế không?
- Câu 102.** Thiết kế schema tracking hành vi người dùng trên website quảng cáo. Volume và query patterns cần cân nhắc?
- Câu 103.** Data lineage và data catalog. Công cụ hỗ trợ: OpenMetadata, DataHub, Amundsen, Google Data Catalog.
- Câu 104.** Idempotency trong data pipeline: tại sao cần? Cách implement cho ETL job chạy định kỳ?
- Câu 105.** Cardinality: high-cardinality vs low-cardinality column. Ảnh hưởng đến performance và storage?
- 
- Câu 106.** Schema evolution với Avro / Protobuf / JSON Schema. Backward compatibility vs forward compatibility?
- Câu 107.** Thiết kế data model cho hệ thống marketing attribution: first touch, last touch, multi-touch, data-driven.
- Câu 108.** OLTP vs OLAP: Tại sao không nên chạy báo cáo OLAP trực tiếp trên OLTP database?
- Câu 109.** Data Quality dimensions: Completeness, Accuracy, Consistency, Timeliness, Validity, Uniqueness. Cách đo lường?
- Câu 110.** Thiết kế ERD cho hệ thống quản lý chiến dịch quảng cáo: campaigns, ad groups, ads, keywords, conversions.
- 
- Câu 111.** Wide table vs Normalized tables trong BigQuery. Tại sao BigQuery thường prefer wide table?
- Câu 112.** Aggregate table / Pre-aggregation: khi nào nên tạo aggregate table thay vì query on-the-fly?
- Câu 113.** Surrogate key vs Natural key: phân biệt và khi nào dùng surrogate key trong data warehouse?
- Câu 114.** Junk dimension là gì? Khi nào dùng junk dimension để gom các low-cardinality flags?
- Câu 115.** Role-playing dimension: ví dụ `date\_dim` dùng cho nhiều foreign keys khác nhau trong fact table.
- 
- Câu 116.** Conformed dimension: tại sao quan trọng trong data warehouse để đảm bảo cross-domain reporting?
- Câu 117.** Fact-less fact table: là gì và khi nào dùng? Ví dụ trong hệ thống quảng cáo?
- Câu 118.** Giải thích \*grain\* của fact table. Tại sao việc xác định đúng grain là bước quan trọng nhất trong data modeling?
- Câu 119.** Thiết kế schema cho hệ thống A/B testing: experiments, variants, user assignments, metric events.
- Câu 120.** Data contract là gì? Làm thế nào enforce data contract giữa producer và consumer trong data platform?
- 
- Câu 121.** Event-driven schema design: thiết kế schema cho event streaming (Kafka) vs batch (BigQuery). Khác nhau như thế nào?
- Câu 122.** Inmon vs Kimball approach: So sánh hai trường phái data warehouse design kinh điển.
- Câu 123.** Thiết kế master data management (MDM) cho customer identity resolution: merge records từ nhiều nguồn.

**Câu 124.** Giải thích \*denormalized lookup\*: nhúng dimension values vào fact table. Trade-off giữa storage và query performance?

**Câu 125.** Hub-and-spoke vs Bus architecture trong data warehouse. Ảnh hưởng đến cross-domain analytics?

## V. ETL / ELT & Data Pipeline

**Câu 126.** ETL vs ELT: Tại sao với cloud data warehouse như BigQuery, ELT thường được ưa chuộng hơn?

**Câu 127.** Thiết kế ETL pipeline lấy dữ liệu từ REST API, transform, load vào BigQuery. Mô tả kiến trúc.

**Câu 128.** Dead-letter queue, retry, alert, circuit breaker trong data pipeline. Implement như thế nào?

**Câu 129.** Incremental load vs Full refresh: khi nào dùng cái nào? Implement incremental load dựa trên `updated\_at`?

**Câu 130.** Change Data Capture (CDC): timestamp-based, trigger-based, log-based (Debezium). Ưu/nhược?

**Câu 131.** Xử lý duplicate data trong pipeline: nguyên nhân và cách phòng ngừa/xử lý?

**Câu 132.** Data backfill: chiến lược backfill lịch sử mà không ảnh hưởng pipeline production?

**Câu 133.** Exactly-once, at-least-once, at-most-once delivery semantics. Cái nào khó đạt nhất và tại sao?

**Câu 134.** Watermarking trong stream processing: tại sao cần khi có late events?

**Câu 135.** Thiết kế pipeline xử lý log access 100M events/ngày tính DAU, WAU, MAU.

**Câu 136.** Fan-out và fan-in pattern trong data pipeline. Ví dụ thực tế?

**Câu 137.** JSON, CSV, Avro, Parquet, ORC: So sánh về kích thước, tốc độ, schema evolution support.

**Câu 138.** Apache Airflow: DAG, Task, Operator, XCom, Sensor, TaskGroup. Xử lý task failure và retry.

**Câu 139.** Viết Airflow DAG: mỗi ngày lúc 2AM extract từ MySQL, transform Python, load lên BigQuery.

**Câu 140.** dbt (data build tool): models, sources, tests, snapshots, macros. Khác traditional ETL như thế nào?

**Câu 141.** Kafka cơ bản: Topic, Partition, Consumer Group, Offset. Tại sao tăng partition tăng throughput?

**Câu 142.** GCP Pub/Sub: So sánh với Apache Kafka. Khi nào dùng Pub/Sub thay vì self-hosted Kafka?

**Câu 143.** Thiết kế pipeline real-time ad click fraud detection: latency < 1 giây, volume 10,000 events/giây.

**Câu 144.** Monitoring data pipeline: metrics quan trọng, công cụ, cách setup alert khi pipeline fail?

**Câu 145.** Giải thích \*pipeline orchestration\*. Khi pipeline có hàng trăm tasks, quản lý dependencies thế nào?

**Câu 146.** Thiết kế data pipeline có \*schema inference\*: tự động detect schema từ JSON API và tạo BigQuery table.

**Câu 147.** Giải thích \*data lineage tracking\* tích hợp trong pipeline. Công cụ: OpenLineage, Marquez.

**Câu 148.** Xử lý NULL values trong pipeline: khi nào drop, khi nào impute, khi nào giữ nguyên? Tại sao quan trọng?

**Câu 149.** Giải thích \*data freshness\*: SLA là data phải cập nhật trong 2 giờ. Cách enforce và monitor?

**Câu 150.** Thiết kế pipeline \*event deduplication\*: nhận event từ nhiều nguồn, đảm bảo mỗi event chỉ được xử lý 1 lần.

**Câu 151.** Streaming join: join 2 Kafka stream. Các loại join: inner, left, temporal. Thách thức là gì?

- Câu 152.** Giải thích *\*Kappa Architecture\** với ví dụ cụ thể: xây dựng toàn bộ data platform dùng streaming only.
- Câu 153.** Viết Python class ``BaseExtractor`` abstract với method ``extract(start_date, end_date)``. Implement cho MySQL và REST API.
- Câu 154.** Giải thích *\*pipeline versioning\**: khi logic transform thay đổi, cần reprocess historical data như thế nào?
- Câu 155.** Apache Airflow vs Prefect vs Dagster vs Mage: so sánh về developer experience, features, và maturity.
- Câu 156.** Thiết kế multi-tenant ETL pipeline: cùng một pipeline chạy cho nhiều clients với config khác nhau.
- Câu 157.** Giải thích *\*data vault loading patterns\**: raw vault load, business vault, information mart.
- Câu 158.** Xử lý timezone trong data pipeline: UTC vs local time. Best practice khi lưu timestamp trong BigQuery?
- Câu 159.** Viết Python code parse và validate nested JSON từ ad platform API (Google Ads, Meta Ads) trước khi load vào BigQuery.
- Câu 160.** Giải thích *\*pipeline DAG dependency\**: task A phụ thuộc task B đang chạy hàng ngày. Cách quản lý cross-DAG dependency?
- Câu 161.** Thiết kế *\*hot/cold data tiering\** cho data pipeline: data mới trong BigQuery, data cũ archive xuống GCS Coldline.
- Câu 162.** Giải thích Pub/Sub *\*message ordering\** và *\*message retention\**. Cách đảm bảo xử lý message theo đúng thứ tự?
- Câu 163.** Spark vs Dataflow vs BigQuery SQL: khi nào chọn mỗi công cụ để transform data?
- Câu 164.** Viết Airflow *\*dynamic DAG\** tự động tạo tasks dựa trên danh sách bảng cần sync từ database.
- Câu 165.** Giải thích *\*pipeline testing pyramid\**: unit test, integration test, end-to-end test cho data pipeline.

## VI. Google Cloud Platform (GCP)

- Câu 166.** BigQuery architecture: phân tách compute và storage. Dremel và Colossus. Tại sao query nhanh?
- Câu 167.** BigQuery pricing: On-demand vs Slot-based (flex/standard/enterprise). Khi nào dùng slot reservation?
- Câu 168.** Viết BigQuery query tối ưu: tránh SELECT \*, dùng partition pruning, tránh self-join trên bảng lớn.
- Câu 169.** BigQuery Storage Read API vs Export: khi nào dùng Storage Read API thay vì export ra GCS?
- Câu 170.** Cloud Composer vs Cloud Workflows vs Cloud Scheduler: so sánh use-case từng dịch vụ orchestration.
- Câu 171.** Cloud Run: containerized workloads, concurrency, memory, timeout cho ETL job. Cold start là gì?
- Câu 172.** Cloud Scheduler: trigger Cloud Run để chạy ETL hàng ngày. Xử lý job fail như thế nào?
- Câu 173.** GCS storage classes: Standard, Nearline, Coldline, Archive. Lifecycle management và Object Versioning.
- Câu 174.** Dataflow (Apache Beam): Batch vs Streaming mode. Autoscaling và worker configuration.

- Câu 175.** Pub/Sub + Dataflow + BigQuery: streaming pipeline. Xử lý late data như thế nào?
- 
- Câu 176.** IAM trong GCP: Principle of least privilege. Service Account với quyền tối thiểu. Workload Identity Federation.
- Câu 177.** Secret Manager vs Environment Variables: khi nào dùng Secret Manager? Truy cập từ Cloud Run?
- Câu 178.** Cloud Monitoring và Logging: setup dashboard và alert cho data pipeline. Structured logging.
- Câu 179.** Artifact Registry: lưu Docker image. Tích hợp với Cloud Build CI/CD.
- Câu 180.** Terraform trên GCP: tạo BigQuery dataset, GCS bucket, Cloud Run service. State management.
- 
- Câu 181.** Dataproc vs Dataflow vs BigQuery: khi nào chọn Spark on Dataproc?
- Câu 182.** BigQuery ML: tạo và train model bằng SQL. Use-case phù hợp? Giới hạn so với Vertex AI?
- Câu 183.** Vertex AI: Pipelines, Feature Store, Model Registry. Tích hợp với BigQuery như thế nào?
- Câu 184.** GCP networking: VPC, Firewall rules, Private Google Access. Tại sao cần Private Google Access cho BigQuery?
- Câu 185.** Cloud Build: CI/CD pipeline tự động test và deploy Cloud Run service khi push lên main.
- 
- Câu 186.** BigQuery Omni: query dữ liệu trên AWS S3 hoặc Azure Blob từ BigQuery không cần move data.
- Câu 187.** Giải thích BigQuery \*Information Schema\*: truy vấn metadata về tables, columns, jobs, reservations.
- Câu 188.** Cloud Spanner: khi nào dùng thay vì Cloud SQL hoặc BigQuery? Giải thích globally distributed ACID transactions.
- Câu 189.** Firestore vs Bigtable vs Spanner: so sánh use-case trong data engineering context.
- Câu 190.** Cloud Tasks vs Pub/Sub: phân biệt. Khi nào dùng Cloud Tasks cho asynchronous job dispatch?
- 
- Câu 191.** BigQuery Connected Sheets: giải thích và use-case. Hạn chế về security và governance?
- Câu 192.** Vertex AI Feature Store: tại sao cần? Giải thích online store vs offline store. Tích hợp với training pipeline.
- Câu 193.** Cloud Composer 2 vs Cloud Composer 1: những cải tiến chính. Airflow version và autoscaling.
- Câu 194.** BigQuery \*authorized views\* và \*authorized routines\*: cách share data mà không cần grant quyền trực tiếp vào table.
- Câu 195.** Giải thích \*BigQuery Reservations\* và \*Workload Management\*: slot pools, reservation assignments, idle capacity.
- 
- Câu 196.** GCP Data Catalog: tự động tag BigQuery tables, tìm kiếm data assets, policy tags. Tích hợp với IAM?
- Câu 197.** Cloud Run Jobs vs Cloud Run Services: phân biệt. Khi nào dùng Cloud Run Jobs cho batch ETL?
- Câu 198.** Eventarc: trigger Cloud Run từ Pub/Sub, GCS events, Audit Logs. Use-case event-driven pipeline.
- Câu 199.** BigQuery \*Authorized Datasets\*: chia sẻ dataset giữa các projects mà không expose underlying tables.
- Câu 200.** Giải thích GCP \*Organization Policies\*: enforce resource restrictions ở organization level. Ảnh hưởng đến data pipeline deployment?
- 
- Câu 201.** Cloud Storage FUSE (gcsfuse): mount GCS bucket như filesystem. Use-case và limitations?
- Câu 202.** BigQuery \*table snapshot\* và \*table clone\*: phân biệt, use-case cho testing và rollback.
- Câu 203.** Dataplex: GCP data mesh platform. Giải thích Lakes, Zones, Assets, Tasks, và Data Quality rules.

**Câu 204.** Giải thích *\*BigQuery Continuous Queries\** (preview): streaming SQL queries chạy liên tục trên Pub/Sub.

**Câu 205.** AlloyDB vs Cloud SQL: khi nào chọn AlloyDB? Columnar engine trong AlloyDB giúp analytics thế nào?

## VII. Git, GitFlow & CI/CD

**Câu 206.** GitFlow workflow: main, develop, feature, release, hotfix branches. Khi nào dùng GitFlow vs trunk-based?

**Câu 207.** Git rebase vs merge: phân biệt, ưu/nhược. Khi nào KHÔNG nên dùng rebase?

**Câu 208.** Git cherry-pick: dùng khi nào? Nguy cơ khi cherry-pick nhiều commit?

**Câu 209.** Git stash, reset (soft/mixed/hard), revert. Sự khác biệt giữa `reset` và `revert`?

**Câu 210.** `.gitignore` và `.gitattributes`. Cấu hình `.gitattributes` xử lý line endings cross-platform?

**Câu 211.** Git hooks: pre-commit, commit-msg, pre-push. Dùng husky hoặc pre-commit framework enforce code quality.

**Câu 212.** Semantic versioning (SemVer): MAJOR.MINOR.PATCH. Tích hợp với CI/CD để auto-tag release.

**Câu 213.** CI/CD pipeline cho data pipeline: lint, test, build docker, push, deploy. Công cụ: GitHub Actions, Cloud Build.

**Câu 214.** Infrastructure as Code (IaC): tại sao version control infrastructure? Terraform vs Pulumi.

**Câu 215.** Testing strategy cho data pipeline: unit test, integration test, data quality test. Công cụ phù hợp.

**Câu 216.** Canary deployment vs Blue/Green deployment. Áp dụng cho Cloud Run service.

**Câu 217.** Docker multi-stage build: tại sao dùng? Viết Dockerfile tối ưu cho Python ETL application.

**Câu 218.** Secrets management trong CI/CD: lưu và truy cập secrets an toàn trong GitHub Actions hoặc Cloud Build.

**Câu 219.** Monorepo vs Polyrepo: ưu/nhược điểm khi quản lý nhiều data pipeline projects.

**Câu 220.** Code review best practices: checklist khi review data pipeline PR.

**Câu 221.** Giải thích `git bisect`: tìm commit gây ra bug bằng binary search. Áp dụng trong data pipeline regression?

**Câu 222.** Giải thích *\*conventional commits\**: format commit message chuẩn. Tại sao quan trọng với auto-changelog?

**Câu 223.** GitHub Actions: viết workflow tự động chạy pytest khi có PR, generate coverage report, comment vào PR.

**Câu 224.** Giải thích *\*trunk-based development\**: developer commit thẳng vào main với feature flags. Khác GitFlow thế nào?

**Câu 225.** Giải thích *\*GitOps\**: dùng Git là single source of truth cho infrastructure. Công cụ: ArgoCD, Flux.

**Câu 226.** Container registry security: scan Docker image tìm CVE với Trivy hoặc Container Analysis. Integrate vào CI?

**Câu 227.** Giải thích *\*dependency management\** trong Python: `requirements.txt`, `pyproject.toml`, `poetry`, `uv`. Best practices.

**Câu 228.** Viết GitHub Actions workflow: khi merge vào main, tự động build Docker image, push lên Artifact Registry, deploy lên Cloud Run.

**Câu 229.** Giải thích *\*release train\** vs *\*continuous deployment\**. Khi nào phù hợp với team data engineering?

**Câu 230.** Pre-commit hooks: ``black``, ``isort``, ``flake8``, ``mypy``. Tự động format và lint code trước khi commit.

## VIII. Looker & Data Visualization

**Câu 231.** LookML: view, model, explore, dimension, measure, join. Giải thích từng khái niệm cơ bản.

**Câu 232.** Phân biệt ``dimension``, ``measure``, ``dimension_group``, ``filter`` trong LookML. Ví dụ với dữ liệu quảng cáo.

**Câu 233.** Looker Derived Tables: Native vs SQL vs Persistent Derived Table (PDT). Khi nào dùng PDT?

**Câu 234.** Tối ưu LookML giảm chi phí BigQuery: Aggregate Awareness là gì và cách implement?

**Câu 235.** Looker Explores và Joins: các loại join, fan trap và chasm trap là gì? Cách tránh?

**Câu 236.** Looker Dashboards: Look vs Dashboard. Share và embed Looker vào ứng dụng (Embedded Analytics)?

**Câu 237.** Looker API: tạo/update content, chạy queries, extract data programmatically. Use-case thực tế?

**Câu 238.** Looker git integration: tại sao LookML được version control giống code?

**Câu 239.** Looker data tests và assertions: viết test trong LookML để đảm bảo data quality?

**Câu 240.** So sánh Looker với Power BI và Tableau về semantic layer, governance, và GCP ecosystem fit.

**Câu 241.** Thiết kế Looker dashboard cho CMO: chỉ phí quảng cáo, CPC, CPM, CTR, ROAS theo kênh và thời gian.

**Câu 242.** Looker Studio vs Looker: khi nào dùng cái nào? Sự khác biệt về governance và scalability.

**Câu 243.** Giải thích *\*Looker semantic layer\**: tại sao định nghĩa metric một lần trong LookML tốt hơn define trong BI tool?

**Câu 244.** LookML ``label``, ``description``, ``group_label``, ``hidden``: cách organize và document metrics cho business users.

**Câu 245.** Giải thích Looker *\*access filters\** và *\*user attributes\**: implement row-level security theo user location.

**Câu 246.** Looker *\*liquid templating\**: dùng Liquid trong LookML để dynamic dimension/measure. Ví dụ?

**Câu 247.** Giải thích Looker *\*refinements\**: extend LookML models mà không sửa code gốc. Use-case?

**Câu 248.** Looker Marketplace: tìm và install block. Ví dụ Google Analytics 4 block tiết kiệm công triển khai thế nào?

**Câu 249.** Viết LookML dimension tính *\*days\_since\_last\_order\** dùng ``datediff``. Tại sao cần type: duration?

**Câu 250.** Looker Scheduler và Alerts: setup tự động gửi báo cáo qua email hoặc Slack khi metric vượt ngưỡng.

## IX. AWS, Redshift & Treasure Data

**Câu 251.** Amazon Redshift architecture: leader node vs compute nodes. Redshift Spectrum. Khi nào dùng thay BigQuery?

**Câu 252.** Redshift distribution styles: EVEN, KEY, ALL, AUTO. Cách chọn distribution key?

- Câu 253.** Redshift sort keys: Compound vs Interleaved. VACUUM và ANALYZE trong Redshift.
- Câu 254.** Redshift Serverless vs Provisioned Clusters: trade-offs chi phí và hiệu năng.
- Câu 255.** AWS Glue: Glue Catalog, Crawler, ETL Jobs, Glue Studio. So sánh với GCP Dataflow.
- 
- Câu 256.** AWS S3 vs GCS: so sánh về tính năng, pricing, integration với data warehouse.
- Câu 257.** Treasure Data (TD): kiến trúc Customer Data Platform (CDP). TD Workflow, TD Query (Presto/Hive), TD Bulk Load.
- Câu 258.** TD Segment Builder và Audience Builder: tạo audience segment từ dữ liệu hành vi người dùng.
- Câu 259.** Tích hợp Treasure Data với hệ thống quảng cáo (Google Ads, Facebook Ads): luồng dữ liệu.
- Câu 260.** Amazon Kinesis vs Kafka vs GCP Pub/Sub: so sánh throughput, latency, managed service trade-offs.
- 
- Câu 261.** AWS Lake Formation: quản lý data lake permissions ở column/row level. So sánh với GCP Data Catalog.
- Câu 262.** AWS Step Functions vs Apache Airflow vs Cloud Composer: so sánh orchestration options.
- Câu 263.** Amazon EMR vs Dataproc: managed Spark clusters. So sánh pricing và feature parity.
- Câu 264.** Redshift COPY command: load data từ S3 vào Redshift. Tối ưu với WLM (Workload Management)?
- Câu 265.** AWS DMS (Database Migration Service): migrate từ on-premise Oracle sang Redshift. Thách thức gì?
- 
- Câu 266.** Treasure Data \*TD Agent\* (Fluentd): collect và ship logs từ server lên TD. Cấu hình như thế nào?
- Câu 267.** Giải thích \*Presto/Trino\* dùng trong Treasure Data. Khi nào query Presto nhanh hơn Hive?
- Câu 268.** AWS Athena: query trực tiếp S3 dùng SQL. So sánh với BigQuery Omni và Redshift Spectrum.
- Câu 269.** TD Workflow: viết workflow YAML định nghĩa multi-step data pipeline trong Treasure Data.
- Câu 270.** Giải thích \*CDP (Customer Data Platform)\*: các thành phần chính và cách TD platform fit vào kiến trúc?

## X. Big Data & Distributed Systems

- Câu 271.** CAP theorem: Consistency, Availability, Partition Tolerance. Ví dụ hệ thống ưu tiên CP và AP.
- Câu 272.** Apache Spark: RDD, DataFrame, Dataset. Tại sao DataFrame được ưu tiên hơn RDD trong Spark hiện đại?
- Câu 273.** Spark execution model: Driver, Executor, Job, Stage, Task. Quá trình shuffle và tại sao tốn kém.
- Câu 274.** Spark tối ưu: Broadcast join, partition pruning, predicate pushdown, caching/persist.
- Câu 275.** Spark Structured Streaming: micro-batch vs continuous processing. Watermarking và stateful operations.
- 
- Câu 276.** Data skew trong Spark: nguyên nhân và cách xử lý (salting, broadcast, repartition).
- Câu 277.** Parquet: row group, column chunk, page. Tại sao tốt cho analytical workloads? Predicate pushdown?
- Câu 278.** Iceberg, Delta Lake, Hudi: so sánh ba table format. ACID transactions trên data lake.
- Câu 279.** Distributed caching: Redis vs Memcached. Khi nào dùng cache trong data pipeline?
- Câu 280.** Consistent hashing: dùng trong distributed systems. Tại sao tốt hơn modulo hashing?
-

- Câu 281.** MapReduce: Map, Shuffle, Reduce. Hạn chế so với Spark?
- Câu 282.** Data skewness trong thống kê và distributed computing. Cách detect và handle?
- Câu 283.** Event sourcing vs traditional CRUD: ưu/nhược. Khi nào nên dùng?
- Câu 284.** Micro-batching vs True Streaming: Spark Structured Streaming, Flink, Dataflow về latency và throughput.
- Câu 285.** Columnar storage vs Row storage: tại sao columnar tốt hơn cho OLAP? I/O savings cụ thể.
- Câu 286.** Giải thích \*vector clock\* và \*lambport timestamp\* trong distributed systems. Dùng để làm gì?
- Câu 287.** Two Generals Problem và Byzantine Fault Tolerance: tại sao không thể có 100% reliable messaging?
- Câu 288.** Paxos vs Raft consensus algorithms: giải thích ý tưởng cơ bản của Raft. Dùng trong Kafka, etcd như thế nào?
- Câu 289.** Giải thích \*backpressure\* trong stream processing. Cách Kafka và Flink xử lý backpressure?
- Câu 290.** Giải thích \*exactly-once\* trong Kafka: idempotent producer, transactional API. Overhead so với at-least-once?
- Câu 291.** Flink vs Spark Streaming: so sánh về true streaming, state management, và event time processing.
- Câu 292.** Apache Iceberg table format: hidden partitioning, time travel, schema evolution. So sánh với Delta Lake.
- Câu 293.** Giải thích \*compaction\* trong data lake: tại sao cần compact small files? Công cụ và trigger strategy.
- Câu 294.** Giải thích \*Z-ordering\* trong Delta Lake / Iceberg. Tại sao giúp data skipping hiệu quả hơn?
- Câu 295.** Distributed transactions: 2PC (Two-Phase Commit) vs Saga pattern. Khi nào dùng Saga cho data pipeline?
- Câu 296.** Giải thích \*CRDT\* (Conflict-free Replicated Data Type): use-case trong distributed systems?
- Câu 297.** Bloom filter: giải thích và ứng dụng trong Parquet, HBase, Kafka để tăng performance.
- Câu 298.** Giải thích \*hot partition problem\* trong Kafka. Cách chọn partition key tốt để phân phối đều?
- Câu 299.** Apache Hudi: Copy-on-Write vs Merge-on-Read table types. Trade-off giữa read và write performance?
- Câu 300.** Giải thích \*lakehouse\* concept: kết hợp data lake và data warehouse. Tại sao đây là xu hướng hiện tại?

## XI. System Design cho Data Engineer

- Câu 301.** Thiết kế data platform cho công ty quảng cáo: 1 tỷ ad impressions/ngày, cần real-time và batch reporting.
- Câu 302.** Thiết kế pipeline đồng bộ dữ liệu từ 10 nguồn (MySQL, Oracle, REST API, FTP, S3) vào BigQuery.
- Câu 303.** BigQuery query đang chạy 30 phút, chi phí cao. Quy trình debug và tối ưu từng bước.
- Câu 304.** Pipeline fail lúc nửa đêm, data không load vào BigQuery. Quy trình xử lý incident.
- Câu 305.** Thiết kế hệ thống tracking conversion cho chiến dịch quảng cáo: click purchase, nhiều touch points, deduplication.
- Câu 306.** Dữ liệu trong Looker dashboard sai lệch so với source. Quy trình điều tra và fix root cause.

- Câu 307.** Thiết kế data retention policy: raw data 90 ngày, aggregated 2 năm. GCS Lifecycle + BigQuery partitioning.
- Câu 308.** Thiết kế hệ thống cho phép business user chạy ad-hoc query mà không ảnh hưởng production pipeline.
- Câu 309.** Multi-tenancy trong data platform: cô lập dữ liệu và compute resource cho nhiều clients/teams.
- Câu 310.** Disaster recovery cho data pipeline: RTO và RPO. Backup và recovery strategy cho BigQuery và GCS.
- Câu 311.** Thiết kế A/B testing data pipeline: phân nhóm users, track metrics, tính statistical significance.
- Câu 312.** Rate limiting và throttling khi gọi external API: exponential backoff, respect API limits.
- Câu 313.** Push-based vs pull-based data pipeline: ưu/nhược điểm từng cách?
- Câu 314.** Thiết kế data quality monitoring: tự động phát hiện anomaly (volume drop, null spike, distribution shift).
- Câu 315.** Thiết kế \*event-driven data platform\*: mỗi khi data thay đổi, trigger downstream pipeline tự động.
- Câu 316.** Thiết kế hệ thống \*real-time recommendation\* cho quảng cáo: từ click event đến serving recommendation < 100ms.
- Câu 317.** Thiết kế \*data sharing platform\*: nhiều team/partner có thể query data của nhau với access control chặt chẽ.
- Câu 318.** Thiết kế pipeline tự động \*detect schema drift\* và alert khi schema nguồn thay đổi không báo trước.
- Câu 319.** Giải thích \*exactly-once data platform\*: từ source extraction đến destination write, không mất và không duplicate.
- Câu 320.** Thiết kế \*cross-cloud data pipeline\*: data ở AWS S3, processing ở GCP Dataflow, result vào BigQuery và Redshift.
- Câu 321.** Thiết kế \*data lake governance\*: access control, data classification, PII detection, và audit trail.
- Câu 322.** Thiết kế hệ thống \*cost attribution\*: theo dõi chi phí BigQuery, GCS, Dataflow cho từng team/project.
- Câu 323.** Thiết kế \*metadata-driven pipeline framework\*: config-driven ETL không cần viết code cho mỗi nguồn mới.
- Câu 324.** Khi có 50 data sources mỗi nguồn có schedule và SLA khác nhau, quản lý dependencies và alerting thế nào?
- Câu 325.** Thiết kế \*hot storage vs cold storage\* strategy: query-ready data trong BigQuery, archive trong GCS với lifecycle policy.

## XII. API Integration & Web Services

- Câu 326.** REST vs GraphQL vs gRPC: so sánh use-case, performance, phù hợp với data engineering.
- Câu 327.** OAuth 2.0 flows: Authorization Code, Client Credentials, Device Code. Khi nào dùng Client Credentials?
- Câu 328.** Viết Python code gọi Google Ads API lấy campaign performance metrics, xử lý pagination, lưu vào BigQuery.
- Câu 329.** Webhook vs Polling: phân biệt. Khi nào dùng webhook để nhận data từ external system?

**Câu 330.** Rate limiting: xử lý HTTP 429. Implement retry với exponential backoff và jitter.

**Câu 331.** API versioning strategies: URL, Header, Query parameter. Ảnh hưởng đến backward compatibility?

**Câu 332.** Idempotent HTTP methods: PUT và DELETE idempotent còn POST thì không. Tại sao quan trọng?

**Câu 333.** Thiết kế Python API wrapper class: authentication, retry, rate limiting, logging, error handling.

**Câu 334.** OpenAPI/Swagger specification: lợi ích khi tài liệu hóa API cho data pipeline integration.

**Câu 335.** Giải thích CORS và ảnh hưởng đến API calls. Liên quan đến data engineering như thế nào?

**Câu 336.** Giải thích \*pagination strategies\*: offset-based, cursor-based, keyset pagination. Cái nào tốt hơn cho large datasets?

**Câu 337.** Viết code consume Google Analytics 4 Data API: authentication, report request, handle quota limits.

**Câu 338.** Giải thích \*webhook reliability\*: retry strategy khi webhook endpoint down. Dead letter, idempotency key.

**Câu 339.** Tích hợp Meta Ads (Facebook) API: Business SDK, rate limits, versioning policy. Thách thức thực tế?

**Câu 340.** Giải thích \*service mesh\* (Istio/Envoy): quan trọng trong microservices data platform. Liên quan đến retry và circuit breaker?

**Câu 341.** Viết Python code dùng `httpx` async để call nhiều ad platform APIs đồng thời: Google Ads, Meta, TikTok Ads.

**Câu 342.** Giải thích \*API gateway pattern\*: Kong, Apigee. Use-case trong data engineering để rate limit và auth external APIs.

**Câu 343.** Giải thích \*long polling\* vs \*WebSocket\* vs \*Server-Sent Events\*. Khi nào dùng cái nào để nhận real-time data?

**Câu 344.** Giải thích \*JSON:API\*, \*HAL\*, \*ODATA\* specifications. Tại sao standardized API format giúp data integration dễ hơn?

**Câu 345.** Viết Python code tự động handle token refresh (OAuth 2.0 refresh token flow) khi access token hết hạn.

### XIII. Monitoring, Observability & Performance

**Câu 346.** Ba pillars of observability: Metrics, Logs, Traces. Công cụ GCP cho từng pillar.

**Câu 347.** SLA, SLO, SLI: phân biệt. Ví dụ với data pipeline: SLO 99.5% freshness trong 2 giờ.

**Câu 348.** OpenTelemetry: add tracing vào data pipeline để track latency của từng step.

**Câu 349.** Alerting fatigue: cách thiết kế alert strategy tránh false positives mà không bỏ sót incident thật.

**Câu 350.** Profiling Python code: `cProfile`, `py-spy`, `memory\_profiler`. Tìm bottleneck trong ETL script.

**Câu 351.** EXPLAIN và EXPLAIN ANALYZE để diagnose slow queries. Patterns cần tránh.

**Câu 352.** Cost optimization GCP: Committed Use Discounts, Spot VMs, BigQuery slot reservations, GCS lifecycle.

**Câu 353.** Data freshness monitoring: detect khi pipeline delay và data không cập nhật đúng lịch.

**Câu 354.** Runbook và Playbook trong incident management. Bạn sẽ viết runbook cho gì với data pipeline?

**Câu 355.** Load testing cho data pipeline: simulate high load, measure throughput/latency. Công cụ nào?

- Câu 356.** Giải thích \*P99 latency\* vs \*average latency\*. Tại sao P99 quan trọng hơn average cho SLO?
- Câu 357.** Giải thích \*error budget\*: nếu SLO là 99.9%, error budget trong 30 ngày là bao nhiêu phút?
- Câu 358.** Viết Python structured logging cho data pipeline: log với fields như `pipeline\_name`, `execution\_id`, `duration\_ms`.
- Câu 359.** Giải thích \*synthetic monitoring\* vs \*real user monitoring\*. Áp dụng cho data pipeline như thế nào?
- Câu 360.** Dashboard design best practices: RED method (Rate, Errors, Duration) và USE method (Utilization, Saturation, Errors).
- Câu 361.** Giải thích \*flame graph\* trong profiling. Đọc flame graph như thế nào để tìm hotspot?
- Câu 362.** BigQuery \*INFORMATION\_SCHEMA.JOBS\*: query để monitor expensive queries, slot usage, và billing. Viết query cụ thể.
- Câu 363.** Giải thích \*cardinality\* vấn đề trong monitoring systems (Prometheus): tại sao tránh high-cardinality labels?
- Câu 364.** Giải thích \*chaos engineering\* (Netflix): áp dụng cho data platform để test resilience.
- Câu 365.** Thiết kế \*data freshness dashboard\* trong Looker: hiển thị thời gian cập nhật cuối cùng của từng table quan trọng.

## XIV. Security & Compliance trong Data Engineering

- Câu 366.** PII: xử lý và bảo vệ dữ liệu cá nhân trong data pipeline. GDPR và PDPA (Việt Nam) yêu cầu gì?
- Câu 367.** Data masking vs Data tokenization vs Data encryption: phân biệt và use-case phù hợp.
- Câu 368.** Column-level security trong BigQuery: Policy Tags và Data Catalog để kiểm soát truy cập cột nhạy cảm.
- Câu 369.** Row-level security trong BigQuery: RLS để user chỉ thấy data của team/region được phép.
- Câu 370.** Encryption at rest và in transit: BigQuery và GCS handle encryption thế nào? CMEK là gì?
- Câu 371.** SQL injection trong dynamic queries: cách tránh khi build query dynamically trong Python.
- Câu 372.** Audit logging: tại sao cần log ai đã query gì trong BigQuery? Dùng Cloud Audit Logs.
- Câu 373.** Network security: Private endpoints, VPC Service Controls, Cloud Armor. Ngăn data exfiltration.
- Câu 374.** Zero trust security model: áp dụng cho data platform như thế nào?
- Câu 375.** SOC 2, ISO 27001, PCI DSS: data engineer cần biết gì về các compliance framework này?
- Câu 376.** Giải thích \*data classification\*: Public, Internal, Confidential, Restricted. Áp dụng vào BigQuery tagging.
- Câu 377.** Giải thích \*RBAC\* (Role-Based Access Control) và \*ABAC\* (Attribute-Based Access Control). BigQuery dùng loại nào?
- Câu 378.** Giải thích \*credential rotation\*: tự động rotate service account keys và secrets. Best practice tần suất?
- Câu 379.** Giải thích \*DLP (Data Loss Prevention)\* API của GCP: scan và mask PII tự động trong data pipeline.
- Câu 380.** Viết thiết kế \*data access review\* process: hàng quý review và revoke quyền truy cập không cần thiết.

- Câu 381.** Giải thích *\*supply chain attack\** trong data engineering: dependencies, Docker images, và cách mitigate?
- Câu 382.** Giải thích *\*SAST\** và *\*DAST\** trong security testing. Áp dụng vào CI/CD của data pipeline?
- Câu 383.** Xử lý *\*secrets rotation\** trong production pipeline không downtime: chiến lược là gì?
- Câu 384.** Giải thích *\*data residency\** và *\*data sovereignty\**: ảnh hưởng đến thiết kế pipeline cross-region?
- Câu 385.** Implement *\*consent management\** trong data pipeline: chỉ process data của users đã cho phép marketing.

## XV. Containers & Infrastructure

- Câu 386.** Docker: image, container, layer, Dockerfile, .dockerignore. Giải thích build cache hoạt động như thế nào?
- Câu 387.** Docker multi-stage build: viết Dockerfile tối ưu cho Python ETL app (builder stage + runtime stage).
- Câu 388.** Docker networking: bridge, host, overlay network. Khi nào dùng host network?
- Câu 389.** Docker volumes vs bind mounts: phân biệt và khi nào dùng trong data pipeline container?
- Câu 390.** Kubernetes cơ bản: Pod, Deployment, Service, ConfigMap, Secret, Namespace. Khi nào cần K8s thay vì Cloud Run?
- Câu 391.** Kubernetes resource requests và limits: tầm quan trọng khi chạy ETL jobs trên K8s.
- Câu 392.** Giải thích Kubernetes *\*Job\** và *\*CronJob\**. Dùng để chạy batch ETL pipeline. Retry policy?
- Câu 393.** Helm charts: tại sao dùng Helm để manage K8s deployments? Viết values.yaml cho ETL application.
- Câu 394.** GKE (Google Kubernetes Engine) vs Cloud Run: khi nào chọn GKE cho data pipeline workloads?
- Câu 395.** Giải thích *\*sidecar pattern\** trong Kubernetes: dùng để collect logs, proxy, hoặc sync data?
- Câu 396.** Terraform best practices: modules, remote state (GCS backend), workspace, và version pinning.
- Câu 397.** Giải thích *\*immutable infrastructure\**: tại sao không SSH vào server để fix? Liên quan đến data engineering?
- Câu 398.** Giải thích *\*container image scanning\**: Trivy, Snyk. Integrate vào CI/CD pipeline.
- Câu 399.** Giải thích *\*resource quotas\** và *\*LimitRange\** trong Kubernetes namespace. Quản lý cost trong shared cluster?
- Câu 400.** Viết ``docker-compose.yml`` để local development: Airflow + PostgreSQL + Redis. Giải thích từng service.

## XVI. Data Quality & Testing

- Câu 401.** Giải thích data quality testing trong data pipeline: unit tests, schema tests, referential integrity tests.
- Câu 402.** Great Expectations: tạo expectation suite, checkpoint, và data docs. So sánh với dbt tests.
- Câu 403.** dbt tests: ``unique``, ``not_null``, ``accepted_values``, ``relationships``. Viết custom generic test.
- Câu 404.** Giải thích *\*data contract testing\**: consumer-driven contract. Công cụ: Pact, Schemathesis.

- Câu 405.** Thiết kế data quality \*scorecard\*: tổng hợp DQ scores theo table, domain, và trend theo thời gian.
- Câu 406.** Anomaly detection cho data quality: dùng statistical methods (Z-score, IQR) hoặc ML để phát hiện anomaly.
- Câu 407.** Giải thích \*data profiling\*: phân tích distribution, uniqueness, nullability, và patterns của data.
- Câu 408.** Viết dbt snapshot test: đảm bảo số bản ghi không giảm quá 10% so với hôm qua (volume anomaly).
- Câu 409.** Giải thích \*quarantine pattern\*: data không pass quality check thì đưa vào quarantine table để review.
- Câu 410.** Thiết kế \*data observability platform\*: Monte Carlo, Acceldata, hoặc tự build. Các tính năng cốt lõi?
- Câu 411.** Giải thích \*referential integrity\*: tại sao BigQuery không enforce FK? Cách implement bằng dbt tests?
- Câu 412.** Viết pytest test cho hàm transform: mock BigQuery client, kiểm tra output schema và row count.
- Câu 413.** Giải thích \*golden dataset testing\*: so sánh output của pipeline mới vs pipeline cũ trên cùng input.
- Câu 414.** Giải thích \*shadow mode deployment\*: chạy pipeline mới song song với pipeline cũ để compare kết quả.
- Câu 415.** Thiết kế QA checklist trước khi release pipeline mới vào production: các bước cần kiểm tra.

## XVII. Machine Learning Engineering cho Data Engineer

- Câu 416.** Giải thích MLOps: tại sao data engineer cần biết về MLOps? Vai trò data engineer trong ML pipeline.
- Câu 417.** Feature engineering: one-hot encoding, target encoding, embedding. Cách implement trong BigQuery SQL.
- Câu 418.** Giải thích \*feature store\*: tại sao cần? Online store vs offline store. Vertex AI Feature Store vs Feast.
- Câu 419.** Train/validation/test split trong time-series data: tại sao không dùng random split? Walk-forward validation là gì?
- Câu 420.** Giải thích \*data leakage\*: nguyên nhân và cách phát hiện. Tại sao đặc biệt nguy hiểm?
- Câu 421.** Thiết kế pipeline cung cấp training data cho ML model: từ raw events đến feature matrix trong BigQuery.
- Câu 422.** Giải thích \*model versioning\* và \*model registry\*: MLflow, Vertex AI Model Registry. Tại sao cần?
- Câu 423.** Batch inference vs Online inference: thiết kế data pipeline cho mỗi loại.
- Câu 424.** Giải thích \*data drift\* và \*concept drift\*: monitor ML model performance theo thời gian.
- Câu 425.** Vertex AI Pipelines: tạo ML pipeline YAML/Python SDK. Tích hợp với BigQuery và GCS.
- Câu 426.** Giải thích \*ONNX\* (Open Neural Network Exchange): tại sao quan trọng cho model portability?
- Câu 427.** Thiết kế pipeline tự động retrain model khi data drift được phát hiện. Trigger là gì?

- Câu 428.** Giải thích \*A/B testing cho ML models\*: cách route traffic, track metrics, và đánh giá statistical significance.
- Câu 429.** BigQuery ML: CREATE MODEL, ML.PREDICT, ML.EVALUATE. Khi nào BigQuery ML đủ tốt vs khi nào cần Vertex AI?
- Câu 430.** Giải thích \*class imbalance\*: SMOTE, class weights, undersampling. Ảnh hưởng đến training data pipeline?

## XVIII. Networking & Distributed Systems Fundamentals

- Câu 431.** TCP vs UDP: phân biệt. Kafka dùng TCP. Tại sao không dùng UDP cho data streaming?
- Câu 432.** HTTP/1.1 vs HTTP/2 vs HTTP/3: các cải tiến chính. gRPC dùng HTTP/2, lợi ích gì?
- Câu 433.** DNS: giải thích A record, CNAME, TTL. Liên quan đến service discovery trong data platform?
- Câu 434.** TLS/SSL handshake: quá trình diễn ra như thế nào? Tại sao HTTPS quan trọng cho API calls trong pipeline?
- Câu 435.** Giải thích \*load balancer\*: L4 vs L7. Cloud Load Balancing trong GCP dùng loại nào cho Cloud Run?
- Câu 436.** Giải thích \*CDN\* (Content Delivery Network): áp dụng cho static data assets như thế nào?
- Câu 437.** NAT Gateway vs Direct Egress: khi nào dùng NAT cho private Cloud Run service gọi ra internet?
- Câu 438.** Giải thích \*service discovery\*: Cloud Run dùng DNS-based discovery. Kubernetes dùng kube-dns.
- Câu 439.** Giải thích \*network latency\* và \*bandwidth\*: tại sao latency quan trọng hơn bandwidth trong distributed systems?
- Câu 440.** VPC Peering vs Shared VPC vs VPN vs Cloud Interconnect: khi nào dùng cái nào để connect on-premise với GCP?

## XIX. Quảng cáo & Marketing Technology (Domain Knowledge)

- Câu 441.** Giải thích các metrics quảng cáo cơ bản: Impressions, Clicks, CTR, CPC, CPM, CPA, ROAS, ROMI.
- Câu 442.** Attribution models: First Touch, Last Touch, Linear, Time Decay, Data-Driven. Ưu/nhược từng model?
- Câu 443.** Giải thích \*programmatic advertising\*: DSP, SSP, Ad Exchange, DMP. Data engineer xử lý dữ liệu từ đâu?
- Câu 444.** Cookie và Third-party cookie: tại sao third-party cookie bị deprecated? Ảnh hưởng đến tracking và attribution?
- Câu 445.** Giải thích \*ITP (Intelligent Tracking Prevention)\* của Safari và ảnh hưởng đến tracking data.
- Câu 446.** Google Ads API và Meta Ads API: cấu trúc hierarchy của account (Account Campaign AdGroup Ad).
- Câu 447.** Giải thích \*conversion tracking\*: pixel, server-side tracking, và Conversions API (CAPI) của Meta.
- Câu 448.** UTM parameters: utm\_source, utm\_medium, utm\_campaign, utm\_content, utm\_term. Cách parse và analyze.
- Câu 449.** Giải thích \*frequency capping\* và \*reach\*: tại sao cần deduplication khi tính unique reach?

- Câu 450.** Giải thích \*viewability\*: VAST, VPAID, Open Measurement SDK. Dữ liệu viewability được thu thập thế nào?
- 
- Câu 451.** Giải thích \*Customer Journey\*: touchpoints, sessions, và cách model trong data warehouse.
- Câu 452.** Real-time bidding (RTB): quy trình đấu thầu diễn ra trong < 100ms. Data engineer cần xử lý dữ liệu gì?
- Câu 453.** Giải thích \*lookalike audience\*: dùng ML để tìm users tương tự với existing customers. Data pipeline cần gì?
- Câu 454.** Google Analytics 4 (GA4): event-based model vs session-based (Universal Analytics). Export sang BigQuery?
- Câu 455.** Giải thích \*identity resolution\*: match users across devices và channels dùng deterministic vs probabilistic matching.
- 
- Câu 456.** Giải thích \*LTV (Lifetime Value)\*: cách tính và predict LTV. Ứng dụng trong targeting quảng cáo.
- Câu 457.** Giải thích \*incrementality testing\*: đo lường thực sự quảng cáo có tăng doanh số không. Khác A/B test thế nào?
- Câu 458.** Giải thích \*media mix modeling (MMM)\*: phân bổ ngân sách quảng cáo tối ưu. Data requirements là gì?
- Câu 459.** Giải thích \*cross-device tracking\*: user dùng nhiều thiết bị. Graph-based identity resolution hoạt động thế nào?
- Câu 460.** Giải thích Privacy Sandbox của Google (FLEDGE, Topics API, Attribution Reporting API) thay thế third-party cookie.

## XX. Soft Skills & Behavioral Questions

- Câu 461.** Mô tả dự án data engineering phức tạp nhất bạn từng làm. Thách thức lớn nhất và cách giải quyết?
- Câu 462.** Bạn phát hiện dữ liệu sai trong production. Bạn làm gì? Quy trình xử lý và báo cáo?
- Câu 463.** Khi yêu cầu từ business không rõ ràng hoặc có thể dẫn đến thiết kế kỹ thuật không tốt, bạn xử lý thế nào?
- Câu 464.** Prioritize tasks khi có nhiều deadline cùng lúc và resources hạn chế. Cách bạn quyết định?
- Câu 465.** Bạn học công nghệ mới như thế nào? Ví dụ về lần tự học công nghệ hoàn toàn mới trong thời gian ngắn.
- 
- Câu 466.** Đảm bảo code quality trong team: quy trình code review của bạn như thế nào?
- Câu 467.** Một lần không đồng ý với quyết định kỹ thuật của team lead. Bạn xử lý thế nào?
- Câu 468.** Tài liệu hóa data pipeline đủ tốt để người mới có thể hiểu và maintain. Cách bạn làm?
- Câu 469.** Onboard vào dự án mới với codebase phức tạp: chiến lược hiểu hệ thống nhanh nhất là gì?
- Câu 470.** Describe a time you optimized a slow data pipeline under pressure. What was your approach?
- 
- Câu 471.** How do you stay updated with the rapidly evolving data engineering landscape?
- Câu 472.** Làm việc với tài liệu kỹ thuật tiếng Anh hoặc tiếng Nhật không có người hỗ trợ. Bạn xử lý thế nào?
- Câu 473.** Chịu áp lực cao về deadline và chất lượng sản phẩm cùng lúc. Kết quả như thế nào?
- Câu 474.** Kinh nghiệm làm việc trong Agile/Scrum: vai trò của Data Engineer trong sprint planning?

**Câu 475.** Tại sao muốn làm việc trong ngành quảng cáo và marketing technology?

**Câu 476.** Bạn đã từng phải explain technical concept phức tạp cho stakeholder không có background kỹ thuật. Ví dụ?

**Câu 477.** Khi bạn phát hiện technical debt lớn trong codebase, bạn propose cải thiện như thế nào mà không gây disrupt?

**Câu 478.** Bạn làm thế nào để estimate effort cho task data engineering mà bạn chưa từng làm trước đây?

**Câu 479.** Bạn có kinh nghiệm mentor junior engineer không? Cách bạn guide họ trong data engineering?

**Câu 480.** Describe a situation where you had to make a trade-off between data quality and delivery speed.

**Câu 481.** Khi pipeline của bạn break production và bạn chưa biết root cause, bước đầu tiên bạn làm là gì?

**Câu 482.** Bạn xây dựng mối quan hệ với business stakeholders như thế nào để hiểu rõ data requirements?

**Câu 483.** Giải thích làm thế nào bạn sẽ onboard một data source mới (ví dụ: một CRM mới) vào data platform hiện tại.

**Câu 484.** Bạn đã từng phải refactor toàn bộ data pipeline mà không downtime. Cách tiếp cận là gì?

**Câu 485.** Nếu phỏng vấn thành công và join công ty, 90 ngày đầu tiên của bạn sẽ làm gì?

## XXI. Japanese & English Communication

**Câu 486.** JLPT level của bạn? Bạn đã đọc tài liệu kỹ thuật tiếng Nhật chưa? Cho ví dụ cụ thể.

**Câu 487.** TOEIC score của bạn? Đánh giá kỹ năng tiếng Anh kỹ thuật của bạn.

**Câu 488.** Dịch: "

**Câu 489.** Dịch: "

**Câu 490.** Dịch và giải thích: "

**Câu 491.** Chiến lược đọc tài liệu kỹ thuật tiếng Nhật hiệu quả mà không cần dịch từng từ?

**Câu 492.** Explain (in English) the concept of idempotency in data pipelines to a non-technical stakeholder.

**Câu 493.** Explain (in English) why BigQuery uses columnar storage and how it benefits analytical queries.

**Câu 494.** Explain (in English) the difference between a data lake and a data warehouse to a business manager.

**Câu 495.** Write a professional email (in English) to a client explaining that their data pipeline will have a 2-hour delay due to upstream issues.

**Câu 496.** Write a pull request description (in English) for a change that optimizes a BigQuery query from 30 minutes to 2 minutes.

**Câu 497.** Giải thích concept ETL cho một developer Nhật Bản không có background data engineering bằng tiếng Anh đơn giản.

## XXII. Câu hỏi tình huống & Problem Solving

**Câu 498.** BigQuery table của bạn có 500 triệu rows và query P90 là 45 giây. Làm thế nào giảm xuống dưới 5 giây?

- Câu 499.** Airflow DAG của bạn bị stuck: task đang RUNNING đã 3 tiếng nhưng không có progress. Điều tra và fix thế nào?
- Câu 500.** Pipeline load data từ API bên ngoài, hôm nay API trả về empty response. Bạn xử lý tình huống này ra sao?
- Câu 501.** BigQuery billing tháng này tăng đột biến 300% so với tháng trước. Cách điều tra nguyên nhân và fix?
- Câu 502.** Một developer mới commit thẳng credentials (API key) vào Git public repo. Bạn phản ứng thế nào? Các bước khẩn cấp?
- Câu 503.** Data trong dashboard sáng nay thấy revenue âm. Root cause analysis từng bước như thế nào?
- Câu 504.** Pipeline đang chạy ổn 6 tháng, đột nhiên hôm nay fail với lỗi 'Out of Memory'. Nguyên nhân có thể là gì?
- Câu 505.** Stakeholder yêu cầu báo cáo mới phải xong trong 2 ngày, nhưng data source chưa được ingest. Bạn làm gì?
- Câu 506.** Phát hiện có data leak: log file bị upload nhầm lên public GCS bucket. Xử lý incident thế nào?
- Câu 507.** Cần migrate toàn bộ data warehouse từ Redshift sang BigQuery với zero downtime. Kế hoạch chi tiết?
- Câu 508.** API nguồn thay đổi format response mà không thông báo, pipeline fail. Thiết kế pipeline resilient hơn thế nào?
- Câu 509.** Business yêu cầu real-time dashboard (< 5 phút freshness) nhưng hiện tại pipeline chạy batch mỗi giờ. Cách approach?
- Câu 510.** Pipeline xử lý 1 triệu records/ngày, đột nhiên cần xử lý 100 triệu records/ngày do viral campaign. Scale thế nào?
- Câu 511.** Cần add thêm cột vào BigQuery table đang được hàng chục downstream pipelines và dashboards sử dụng. Approach?
- Câu 512.** Bạn join công ty, phát hiện toàn bộ data pipeline không có test và documentation. Bạn sẽ cải thiện từ đâu?
- Câu 513.** Pipeline cần join 2 bảng: fact table 10 tỷ rows và dimension table 1 triệu rows. Cách tối ưu join này?
- Câu 514.** Dữ liệu từ nguồn A và nguồn B cho cùng metric nhưng cho kết quả khác nhau 5%. Điều tra thế nào?
- Câu 515.** Cần implement GDPR right-to-erasure: xóa toàn bộ data của một user khỏi data warehouse. Approach?
- Câu 516.** Nhận được yêu cầu: 'Hãy build thứ gì đó để chúng ta có thể dự đoán ad click-through rate.' Bạn làm gì đầu tiên?
- Câu 517.** Pipeline nhận event từ mobile app, nhưng 20% events đến muộn > 24 giờ. Thiết kế pipeline xử lý late events?

## XXIII. Coding Challenges & Algorithms

- Câu 518.** Viết Python function `merge_intervals(intervals)`: nhận list `[(1,3),(2,6),(8,10)]` trả về `[(1,6),(8,10)]`.
- Câu 519.** Viết Python function tính số lần xuất hiện của mỗi từ trong file text lớn, sử dụng ít memory nhất có thể.

- Câu 520.** Implement một LRU Cache bằng Python mà không dùng `functools.lru_cache`.
- Câu 521.** Viết Python function `flatten(lst)` làm phẳng nested list bất kỳ độ sâu: `[[1,[2,3]],4]]` thành `[1,2,3,4]`.
- Câu 522.** Viết Python generator `chunk(iterable, size)` chia iterable thành chunks kích thước cố định.
- 
- Câu 523.** Viết SQL: từ bảng `events(user_id, event_time, event_type)`, tìm users đã thực hiện event 'purchase' trong vòng 24h sau 'add\_to\_cart'.
- Câu 524.** Viết Python code sort 1 tỷ integers từ file, memory limit 1 GB. Dùng thuật toán gì? (External Sort)
- Câu 525.** Viết Python function detect và remove cycle trong linked list dùng Floyd's algorithm.
- Câu 526.** Viết SQL tìm median salary trong bảng employees mà không dùng hàm MEDIAN().
- Câu 527.** Viết Python function kiểm tra xem một JSON object có match với một JSON Schema đơn giản không.
- 
- Câu 528.** Viết Python function `retry_with_backoff(func, max_retries, base_delay)` với jitter.
- Câu 529.** Viết Python code đọc Parquet file từ GCS, lọc rows theo điều kiện, và ghi kết quả vào BigQuery.
- Câu 530.** Viết SQL query: mỗi user, tìm 3 sản phẩm gần đây nhất họ đã xem, formatted thành một string ngăn cách bởi dấu phẩy.
- Câu 531.** Viết Python script so sánh schema của 2 BigQuery tables và in ra danh sách các cột khác nhau.
- Câu 532.** Implement thread-safe singleton pattern trong Python cho database connection pool.
- 
- Câu 533.** Viết Python function parse log line: `'2024-01-15 10:23:45 ERROR [pipeline] Failed to load table: row_count=0'` thành dict.
- Câu 534.** Viết SQL: từ bảng `transactions(id, user_id, amount, created_at)`, tìm users có 3 giao dịch liên tiếp tăng dần.
- Câu 535.** Viết Python context manager `timer()` đo và log thời gian thực thi của code block.
- Câu 536.** Viết Python code tự động paginate API endpoint và collect tất cả results (có thể có hàng triệu records).
- Câu 537.** Viết SQL dùng window function tính cumulative revenue và ngày đầu tiên revenue vượt mốc 1 triệu đô.